Génération PDF

Besoin d’avoir les polices dans le stockage GCP Cloud Storage

Requirements

pymupdf>=1.23.0
functions-framework==3.*
pypdf>=6.6.0
requests>=2.32.5

Code

```
import fitz  # PyMuPDF
import os
import json
import tempfile
import unicodedata
import textwrap
from datetime import datetime

from flask import Request, send_file, abort
from google.cloud import storage

# ============================================================
# CONFIGURATION
# ============================================================

BUCKET_NAME = "xxxtech-generation-pdf-assets"

TEMP_FONT_DIR = "/tmp/fonts"
os.makedirs(TEMP_FONT_DIR, exist_ok=True)

FONT_REGULAR_PATH = os.path.join(TEMP_FONT_DIR, "Gotham-BookItalic.ttf")
FONT_BOLD_PATH = os.path.join(TEMP_FONT_DIR, "Gotham-Ultra-Italic.ttf")

# Mapping between placeholders found in the PDF and payload keys
PLACEHOLDER_TO_FIELD_MAP = {
    "<<PRENOM>>": "prenom",
    "<<PRÉNOM>>": "prenom",
    "<<NOM>>": "nom",
    "<<ENTREPRISE>>": "entreprise",
```

```
    "<<VOIE>>": "voie",
    "<<TYPE DE": "voie",
    "<<NUM>>": "numero",
    "<<NUMERO>>": "numero",
    "<<NOM TABLE>>": "table",
    "<<ZONE>>":"zone",
}

# Style configuration per field
FIELD_STYLE_CONFIG = {
    "nom": {"bold": True},
    "entreprise": {"bold": True},
    "voie": {"bold": True},
    "numero": {"bold": True},
    "table": {"bold": True},
    "zone": {"bold": True},
}

# ============================================================
# UTILITY FUNCTIONS
# ============================================================

def int_to_rgb(color_int: int) -> tuple[float, float, float]:
    """
    Convert an integer color (PDF format) to normalized RGB tuple.
    """
    return (
        ((color_int >> 16) & 255) / 255,
        ((color_int >> 8) & 255) / 255,
        (color_int & 255) / 255,
    )


def download_from_gcs(bucket_name: str, blob_name: str, destination_path: str) ->
None:
    """
    Download a file from Google Cloud Storage.
    """
    storage_client = storage.Client()
    bucket = storage_client.bucket(bucket_name)
    bucket.blob(blob_name).download_to_filename(destination_path)


def ensure_fonts_are_available() -> None:
    """
    Ensure required fonts are available locally.
    Downloads them from GCS if missing.
    """
    if not os.path.exists(FONT_REGULAR_PATH):
        download_from_gcs(
            BUCKET_NAME,
```

```
            "fonts/Gotham-BookItalic.ttf",
            FONT_REGULAR_PATH,
        )

    if not os.path.exists(FONT_BOLD_PATH):
        download_from_gcs(
            BUCKET_NAME,
            "fonts/Gotham-Ultra-Italic.ttf",
            FONT_BOLD_PATH,
        )


def normalize_filename(value: str) -> str:
    """
    Normalize a string to be safely used as a filename.
    """
    if not value:
        return "unknown"

    normalized = unicodedata.normalize("NFKD", value)
    ascii_value = normalized.encode("ascii", "ignore").decode("ascii")
    sanitized = ascii_value.lower().replace(" ", "_")

    return "".join(char for char in sanitized if char.isalnum() or char == "_")

def smart_wrap(text, max_chars=25):
    """
    Manual wrap and force wrap
    """
    return "\n".join(
        textwrap.wrap(text, width=max_chars, break_long_words=True)
    )

# ============================================================
# CLOUD FUNCTION ENTRY POINT
# ============================================================

def generate_pdf(request: Request):
    """
    Google Cloud Function entry point.
    Generates a personalized badge PDF from a template.
    """

    # ------------------------------------------------------
    # Input validation
    # ------------------------------------------------------

    if "file" not in request.files or "data" not in request.form:
        abort(400, "Missing required 'file' or 'data' field")

    payload = json.loads(request.form["data"])
```

```
    uploaded_pdf = request.files["file"]

    ensure_fonts_are_available()

    # ------------------------------------------------------
    # Temporary workspace
    # ------------------------------------------------------

    with tempfile.TemporaryDirectory() as temp_dir:
        input_pdf_path = os.path.join(temp_dir, "input.pdf")
        output_pdf_path = os.path.join(temp_dir, "output.pdf")

        uploaded_pdf.save(input_pdf_path)

        document = fitz.open(input_pdf_path)
        page = document[0]

        # Register fonts inside the PDF
        page.insert_font(fontname="Regular", fontfile=FONT_REGULAR_PATH)
        page.insert_font(fontname="Bold", fontfile=FONT_BOLD_PATH)

        text_content = page.get_text("dict")

        # ------------------------------------------------------
        # Text replacement loop
        # ------------------------------------------------------

        for block in text_content["blocks"]:
            if block["type"] != 0:  # Only text blocks
                continue

            for line in block["lines"]:
                for span in line["spans"]:
                    original_text = span["text"].strip()

                    if original_text not in PLACEHOLDER_TO_FIELD_MAP:
                        continue

                    payload_key = PLACEHOLDER_TO_FIELD_MAP[original_text]
                    replacement_value = str(payload.get(payload_key, ""))

                    is_bold = FIELD_STYLE_CONFIG.get(payload_key, {}).get("bold",
False)
                    font_name = "Bold" if is_bold else "Regular"

                    x0, y0, x1, y1 = span["bbox"]
                    font_size = span["size"]
                    text_color = int_to_rgb(span["color"])

                    # To raise up the dimension of the block wich hide previous
text
```

```
padding_left = 3
padding_right = 3
padding_top = 1
padding_bottom = 3

erase_box = fitz.Rect(
    x0 - padding_left,
    y0 - padding_top,
    x1 + padding_right,
    y1 + padding_bottom,
)

# erase
page.draw_rect(
    erase_box,
    fill=(1, 1, 1),
    color=None,
    width=0,
    overlay=True,
)

# gestion intelligente : max 2 lignes + réduction font
max_lines = 2
min_font_size = font_size * 0.5
current_font_size = font_size

while True:
    # on adapte légèrement le wrap à la taille de la font
    scale_ratio  = current_font_size / font_size
    dynamic_width = int(18 / scale_ratio * 1.2)

    wrapped_lines = textwrap.wrap(
        replacement_value,
        width=dynamic_width,
        break_long_words=True
    )

    if len(wrapped_lines) <= max_lines:
        break

    current_font_size -= 0.1

    if current_font_size < min_font_size:
        break  # sécurité (évite boucle infinie)

# affichage
line_height = current_font_size * 1.2
y_cursor = y0 + current_font_size

for line in wrapped_lines:
    page.insert_text(
```

```
                    (x0, y_cursor),
                    line,
                    fontsize=current_font_size,
                    fontname=font_name,
                    color=text_color,
                )
                y_cursor += line_height



# ------------------------------------------------------
# Save optimized PDF
# ------------------------------------------------------

document.save(
    output_pdf_path,
    garbage=4,
    deflate=True,
    clean=True,
    incremental=False,
)
document.close()

# ------------------------------------------------------
# Build output filename
# ------------------------------------------------------

first_name = normalize_filename(payload.get("prenom", ""))
last_name = normalize_filename(payload.get("nom", ""))
date_suffix = datetime.now().strftime("%Y%m%d")

output_filename = f"badge_{first_name}_{last_name}_{date_suffix}.pdf"

return send_file(
    output_pdf_path,
    mimetype="application/pdf",
    as_attachment=True,
    download_name=output_filename,
)
```

Merge PDF

Requirements

pymupdf>=1.23.0
functions-framework==3.*
google-cloud-storage>=3.8.0

Code

```
import fitz  # PyMuPDF
import os
import json
import tempfile
import unicodedata
import textwrap
from datetime import datetime

from flask import Request, send_file, abort
from google.cloud import storage

# ============================================================
# CONFIGURATION
# ============================================================

BUCKET_NAME = "xxxtech-generation-pdf-assets"

TEMP_FONT_DIR = "/tmp/fonts"
os.makedirs(TEMP_FONT_DIR, exist_ok=True)

FONT_REGULAR_PATH = os.path.join(TEMP_FONT_DIR, "Gotham-BookItalic.ttf")
FONT_BOLD_PATH = os.path.join(TEMP_FONT_DIR, "Gotham-Ultra-Italic.ttf")

# Mapping between placeholders found in the PDF and payload keys
PLACEHOLDER_TO_FIELD_MAP = {
    "<<PRENOM>>": "prenom",
    "<<PRÉNOM>>": "prenom",
    "<<NOM>>": "nom",
    "<<ENTREPRISE>>": "entreprise",
    "<<VOIE>>": "voie",
    "<<TYPE DE": "voie",
    "<<NUM>>": "numero",
    "<<NUMERO>>": "numero",
    "<<NOM TABLE>>": "table",
    "<<ZONE>>":"zone",
}

# Style configuration per field
FIELD_STYLE_CONFIG = {
```

```
    "nom": {"bold": True},
    "entreprise": {"bold": True},
    "voie": {"bold": True},
    "numero": {"bold": True},
    "table": {"bold": True},
    "zone": {"bold": True},
}

# ============================================================
# UTILITY FUNCTIONS
# ============================================================

def int_to_rgb(color_int: int) -> tuple[float, float, float]:
    """
    Convert an integer color (PDF format) to normalized RGB tuple.
    """
    return (
        ((color_int >> 16) & 255) / 255,
        ((color_int >> 8) & 255) / 255,
        (color_int & 255) / 255,
    )


def download_from_gcs(bucket_name: str, blob_name: str, destination_path: str) ->
None:
    """
    Download a file from Google Cloud Storage.
    """
    storage_client = storage.Client()
    bucket = storage_client.bucket(bucket_name)
    bucket.blob(blob_name).download_to_filename(destination_path)


def ensure_fonts_are_available() -> None:
    """
    Ensure required fonts are available locally.
    Downloads them from GCS if missing.
    """
    if not os.path.exists(FONT_REGULAR_PATH):
        download_from_gcs(
            BUCKET_NAME,
            "fonts/Gotham-BookItalic.ttf",
            FONT_REGULAR_PATH,
        )

    if not os.path.exists(FONT_BOLD_PATH):
        download_from_gcs(
            BUCKET_NAME,
            "fonts/Gotham-Ultra-Italic.ttf",
            FONT_BOLD_PATH,
        )
```

```python
def normalize_filename(value: str) -> str:
    """
    Normalize a string to be safely used as a filename.
    """
    if not value:
        return "unknown"

    normalized = unicodedata.normalize("NFKD", value)
    ascii_value = normalized.encode("ascii", "ignore").decode("ascii")
    sanitized = ascii_value.lower().replace(" ", "_")

    return "".join(char for char in sanitized if char.isalnum() or char == "_")

def smart_wrap(text, max_chars=25):
    """
    Manual wrap and force wrap
    """
    return "\n".join(
        textwrap.wrap(text, width=max_chars, break_long_words=True)
    )

# ============================================================
# CLOUD FUNCTION ENTRY POINT
# ============================================================

def generate_pdf(request: Request):
    """
    Google Cloud Function entry point.
    Generates a personalized badge PDF from a template.
    """

    # ------------------------------------------------------
    # Input validation
    # ------------------------------------------------------

    if "file" not in request.files or "data" not in request.form:
        abort(400, "Missing required 'file' or 'data' field")

    payload = json.loads(request.form["data"])
    uploaded_pdf = request.files["file"]

    ensure_fonts_are_available()

    # ------------------------------------------------------
    # Temporary workspace
    # ------------------------------------------------------

    with tempfile.TemporaryDirectory() as temp_dir:
        input_pdf_path = os.path.join(temp_dir, "input.pdf")
```

```
        output_pdf_path = os.path.join(temp_dir, "output.pdf")

        uploaded_pdf.save(input_pdf_path)

        document = fitz.open(input_pdf_path)
        page = document[0]

        # Register fonts inside the PDF
        page.insert_font(fontname="Regular", fontfile=FONT_REGULAR_PATH)
        page.insert_font(fontname="Bold", fontfile=FONT_BOLD_PATH)

        text_content = page.get_text("dict")

        # ------------------------------------------------------
        # Text replacement loop
        # ------------------------------------------------------

        for block in text_content["blocks"]:
            if block["type"] != 0:  # Only text blocks
                continue

            for line in block["lines"]:
                for span in line["spans"]:
                    original_text = span["text"].strip()

                    if original_text not in PLACEHOLDER_TO_FIELD_MAP:
                        continue

                    payload_key = PLACEHOLDER_TO_FIELD_MAP[original_text]
                    replacement_value = str(payload.get(payload_key, ""))

                    is_bold = FIELD_STYLE_CONFIG.get(payload_key, {}).get("bold",
False)
                    font_name = "Bold" if is_bold else "Regular"

                    x0, y0, x1, y1 = span["bbox"]
                    font_size = span["size"]
                    text_color = int_to_rgb(span["color"])

                    # To raise up the dimension of the block wich hide previous
text
                    padding_left = 3
                    padding_right = 3
                    padding_top = 1
                    padding_bottom = 3

                    erase_box = fitz.Rect(
                        x0 - padding_left,
                        y0 - padding_top,
                        x1 + padding_right,
                        y1 + padding_bottom,
```

```
)

# erase
page.draw_rect(
    erase_box,
    fill=(1, 1, 1),
    color=None,
    width=0,
    overlay=True,
)

# gestion intelligente : max 2 lignes + réduction font
max_lines = 2
min_font_size = font_size * 0.5
current_font_size = font_size

while True:
    # on adapte légèrement le wrap à la taille de la font
    scale_ratio  = current_font_size / font_size
    dynamic_width = int(18 / scale_ratio * 1.2)

    wrapped_lines = textwrap.wrap(
        replacement_value,
        width=dynamic_width,
        break_long_words=True
    )

    if len(wrapped_lines) <= max_lines:
        break

    current_font_size -= 0.1

    if current_font_size < min_font_size:
        break  # sécurité (évite boucle infinie)

# affichage
line_height = current_font_size * 1.2
y_cursor = y0 + current_font_size

for line in wrapped_lines:
    page.insert_text(
        (x0, y_cursor),
        line,
        fontsize=current_font_size,
        fontname=font_name,
        color=text_color,
    )
    y_cursor += line_height
```

```python
# ------------------------------------------------------
# Save optimized PDF
# ------------------------------------------------------

document.save(
    output_pdf_path,
    garbage=4,
    deflate=True,
    clean=True,
    incremental=False,
)
document.close()

# ------------------------------------------------------
# Build output filename
# ------------------------------------------------------

first_name = normalize_filename(payload.get("prenom", ""))
last_name = normalize_filename(payload.get("nom", ""))
date_suffix = datetime.now().strftime("%Y%m%d")

output_filename = f"badge_{first_name}_{last_name}_{date_suffix}.pdf"

return send_file(
    output_pdf_path,
    mimetype="application/pdf",
    as_attachment=True,
    download_name=output_filename,
)
```